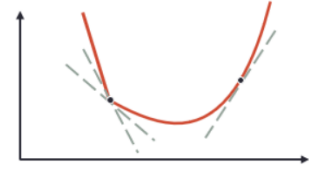


Part A – Optimization

As we saw in Tutorial 08, subgradients generalize gradients to convex functions which are not necessarily differentiable. Notice: you can solve this exercise even before watching Tutorial 08.

Definition: the set of **subgradients** of $f: V \rightarrow \mathbb{R}$ at point $u \in V$ is:

$$\partial f(u) \triangleq \{q \in V \mid \forall v \in V: f(v) \geq f(u) + q^T(v - u)\}.$$



1. Let $f(x) = \begin{cases} x^2, & x < 0 \\ 2x, & x \geq 0 \end{cases}$.

1.1. Is f convex? No need to explain.

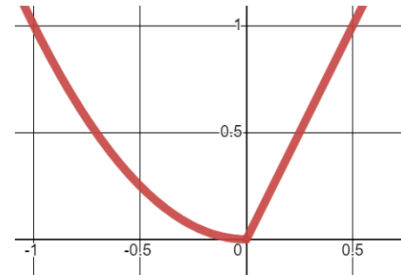
1.2. Propose a sub-derivative function g for f . That is, $g \in \partial f$.

Use the above definition to prove that $g(u) \in \partial f(u), \forall u \in \mathbb{R}$.

1.3. Set a learning rate of $\eta = 0.25$ and a starting point $x_0 = -1.5$.

Running subgradient descent, will the algorithm converge to a minimum?

Prove your answer by filling the following table like we did in Tutorial 07 using as many rows as needed.



i	x_i	$f(x_i)$	$\frac{\partial}{\partial x} f(x_i) = g(x_i)$
0	-1	1	
1			
$\vdots$			

1.4. Repeat 1.3 with $\eta = 1, x_0 = -1.5$.

1.1. Yes.

1.2. let

$$g(u) = \begin{cases} 2u & u \leq 0 \\ 2 & u > 0 \end{cases}$$

Let $u \leq 0$. $f(u) = u^2$ and sign $q = g(u) = 2u$.

Let $v \leq 0$.

$$\begin{aligned} f(v) &\geq f(u) + q(v - u) \\ v^2 &\geq u^2 + 2u(v - u) \\ v^2 - 2uv + u^2 &\geq 0 \\ (v - u)^2 &\geq 0 \end{aligned}$$

And that always true.

If $v > 0$.

$$\begin{aligned} f(v) &\geq f(u) + q(v - u) \\ 2v &\geq u^2 + 2u(v - u) \\ \underbrace{u^2}_{\geq 0} + \underbrace{2v}_{\geq 0} \underbrace{\left(1 - \underbrace{u}_{\leq 0}\right)}_{\geq 0} &\geq 0 \end{aligned}$$

And that always true.

So for $u \leq 0$ we get that $g(u)$ is sub-gradieant.

Now for $u > 0$. $f(u) = 2u$ and $\text{sign } q = g(u) = 2$.

Let $v \leq 0$.

$$\begin{aligned} f(v) &\geq f(u) + q(v - u) \\ v^2 &\geq 2u + 2(v - u) \\ v^2 + \underbrace{(-2v)}_{\geq 0} &\geq 0 \end{aligned}$$

And that always true.

If $v > 0$.

$$\begin{aligned} f(v) &\geq f(u) + q(v - u) \\ 2v &\geq 2u + 2(v - u) \\ 0 &\geq 0 \end{aligned}$$

And that always true.

So for $u > 0$ we get that $g(u)$ is sub-gradieant.

All in all we get that for any $u \in \mathbb{R}$, $g(u)$ is sub-gradient of f . So, $\forall u \in \mathbb{R} \quad g(u) \in \partial f(u)$

1.3. Notice that $x_{i+1} = x_i - \eta \cdot \nabla f(x_i)$

In our case: $\eta = 0.25$, $x_0 = -1.5$.

i	x_i	$f(x_i)$	$\frac{\partial}{\partial x} f(x_i) = g(x_i)$
0	-1.5	2.25	-3
1	$-1.5 - (-0.75) = -0.75$	0.5625	-1.5
2	-0.375	0.14	-0.75
3	-0.1875	0.035	-0.375
4	-0.09	0.008	-0.1875
...	...	...	...

The algorithm will converge to 0 after many more iterations.

1.4. Now our case is : $\eta = 1$, $x_0 = -1.5$.

i	x_i	$f(x_i)$	$\frac{\partial}{\partial x} f(x_i) = g(x_i)$
0	-1.5	2.25	-3
1	1.5	3	2
2	-0.5	0.25	-1

3	0.5	1	2
4	-1.5	2.25	-3
5	1.5	3	2
6	-0.5	0.25	-1
7	0.5	1	2
...	...	...	...

This algorithm will never converge. It easy to see that it repeats steps 0-3 all over again every 4 steps, which prevent it to converge.

Part B – Regression

2. This exercise will investigate the regularization coefficient λ as it was presented in the ridge linear regression section of this course. Suppose we are trying to fit a polynomial to the following data:

X	Y
0	0
1	3
2	12

Our hypothesis class for this problem will be

$$\mathcal{H} = \{w_0 + w_1x + w_2x^2 + w_3x^3 : (w_0, w_1, w_2, w_3) \in \mathbb{R}^4\}.$$

- 2.1. Show that we can fit the data with $w_0 = 0, w_1 = 2, w_2 = 0, w_3 = 1$.
2.2. Show that our hypothesis class is too expressive for the problem we're dealing with. In other words, find a simple quadratic polynomial that fits the data perfectly.
2.3. Denote the mean squared error (MSE)

$$\mathcal{L}(w) = \frac{1}{m} \|Xw - y\|_2^2,$$

Where X is the appropriate Vandermonde matrix.

Calculate $\mathcal{L}(w)$ for the quadratic model in (2.2) and the cubic model in (2.1).

- 2.4. The best line for fitting the data is $y = 6x - 1$. Calculate $\mathcal{L}(w)$ for this line.
2.5. Now denote the MSE with regularization as show in class

$$\mathcal{L}_\lambda(w) = \frac{1}{m} \|Xw - y\|_2^2 + \lambda \|w\|_2^2.$$

Here $\lambda > 0$ is a hyperparameter, which is not given. As we learned in class, the regularization imposes a "cost" on models with large coefficients. Calculate $\mathcal{L}_\lambda(w)$ for each of the three models in (2.1), (2.2) and (2.4).

- 2.6. As it turns out, $\mathcal{L}_\lambda(w)$ would never prefer the simple quadratic polynomial over the cubic polynomial we found, no matter the value of $\lambda > 0$. Can you explain why?
2.7. Suggest a way to fix the regularization method to prefer the model we consider to be simpler.

2.1.

$$\begin{aligned}f(x) &= w_0 + w_1x + w_2x^2 + w_3x^3 = 2x + x^3 \\f(0) &= 0 \\f(1) &= 2 + 1 = 3 \\f(2) &= 4 + 8 = 12\end{aligned}$$

2.2.

$$\begin{aligned}g(x) &= 3x^3 \\g(0) &= 0 \\g(1) &= 3 \\g(2) &= 12\end{aligned}$$

2.3 For the cubic in 2.1:

$$X = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \end{pmatrix}, \quad w = \begin{pmatrix} 0 \\ 2 \\ 0 \\ 1 \end{pmatrix}, \quad y = \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix}, \quad m = 3$$

$$\frac{1}{m} \|Xw - y\|_2^2 = \frac{1}{m} \left\| \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \end{pmatrix} \begin{pmatrix} 0 \\ 2 \\ 0 \\ 1 \end{pmatrix} - \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} \right\|_2^2 = \frac{1}{m} \left\| \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} - \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} \right\|_2^2 = \frac{1}{m} \cdot 0 = 0$$

So it fits the data perfectly.

For the quadratic in 2.2:

$$X = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \end{pmatrix}, \quad w = \begin{pmatrix} 0 \\ 0 \\ 3 \\ 0 \end{pmatrix}, \quad y = \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix}, \quad m = 3$$

$$\frac{1}{m} \|Xw - y\|_2^2 = \frac{1}{m} \left\| \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 3 \\ 0 \end{pmatrix} - \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} \right\|_2^2 = \frac{1}{m} \left\| \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} - \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} \right\|_2^2 = \frac{1}{m} \cdot 0 = 0$$

So it fits the data perfectly.

2.4. For the line $y = 6x - 1$ we get the loss:

$$\begin{aligned} \frac{1}{m} \|Xw - y\|_2^2 &= \frac{1}{m} \left\| \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \end{pmatrix} \begin{pmatrix} -1 \\ 6 \\ 0 \\ 0 \end{pmatrix} - \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} \right\|_2^2 = \frac{1}{m} \left\| \begin{pmatrix} -1 \\ 5 \\ 11 \end{pmatrix} - \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} \right\|_2^2 \\ &= \frac{1}{m} \cdot \left\| \begin{pmatrix} -1 \\ 2 \\ 1 \end{pmatrix} \right\|_2^2 = \frac{1}{m} (1 + 4 + 1) = \frac{1}{3} \cdot 6 = 2 \end{aligned}$$

2.5. For $\lambda > 0$ we get:

$$\mathcal{L}_\lambda(w) = \frac{1}{m} \|Xw - y\|_2^2 + \lambda \|w\|_2^2$$

For the cubic in 2.1:

$$X = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \end{pmatrix}, \quad w = \begin{pmatrix} 0 \\ 2 \\ 0 \\ 1 \end{pmatrix}, \quad y = \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix}, \quad m = 3$$

$$\begin{aligned} \frac{1}{m} \|Xw - y\|_2^2 + \|w\|_2^2 &= \frac{1}{m} \left\| \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \end{pmatrix} \begin{pmatrix} 0 \\ 2 \\ 0 \\ 1 \end{pmatrix} - \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} \right\|_2^2 + \lambda \cdot \left\| \begin{pmatrix} 0 \\ 2 \\ 0 \\ 1 \end{pmatrix} \right\|_2^2 \\ &= \frac{1}{m} \left\| \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} - \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} \right\|_2^2 + \lambda \cdot (4 + 1) = 0 + 5\lambda = 5\lambda \end{aligned}$$

For the quadratic in 2.2:

$$X = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \end{pmatrix}, \quad w = \begin{pmatrix} 0 \\ 0 \\ 3 \\ 0 \end{pmatrix}, \quad y = \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix}, \quad m = 3$$

$$\begin{aligned} \frac{1}{m} \|Xw - y\|_2^2 + \|w\|_2^2 &= \frac{1}{m} \left\| \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 3 \\ 0 \end{pmatrix} - \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} \right\|_2^2 + \lambda \cdot \left\| \begin{pmatrix} 0 \\ 0 \\ 3 \\ 0 \end{pmatrix} \right\|_2^2 \\ &= \frac{1}{m} \left\| \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} - \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} \right\|_2^2 + \lambda \cdot (9) = 0 + 9\lambda = 9\lambda \end{aligned}$$

For the line in 2.4:

$$X = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \end{pmatrix}, \quad w = \begin{pmatrix} -1 \\ 6 \\ 0 \\ 0 \end{pmatrix}, \quad y = \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix}, \quad m = 3$$

$$\begin{aligned} \frac{1}{m} \|Xw - y\|_2^2 + \|w\|_2^2 &= \frac{1}{m} \left\| \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \end{pmatrix} \begin{pmatrix} -1 \\ 6 \\ 0 \\ 0 \end{pmatrix} - \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} \right\|_2^2 + \lambda \cdot \left\| \begin{pmatrix} -1 \\ 6 \\ 0 \\ 0 \end{pmatrix} \right\|_2^2 \\ &= \frac{1}{m} \left\| \begin{pmatrix} -1 \\ 5 \\ 11 \end{pmatrix} - \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} \right\|_2^2 + \lambda \cdot (1 + 37) = \frac{1}{m} \cdot \left\| \begin{pmatrix} -1 \\ 2 \\ 1 \end{pmatrix} \right\|_2^2 + \lambda \cdot (37) \\ &= \frac{1}{3} \cdot 6 + 37\lambda = 2 + 37\lambda \end{aligned}$$

2.6. the loss of the quadratic is 9λ while the loss of the cubic is 5λ . We know that $\lambda > 0$ so it maintains that $5\lambda < 9\lambda$ for any λ . By this $\mathcal{L}_\lambda(w)$ will never prefer the quadratic polynomial than the cubic one, no matter what $\lambda > 0$ is, because its loss is always higher.

2.7. Fixing regularization model to prefer models that we consider to be simpler, means that we want the model to be with lower coefficients on high powers of x in the polynomial. To fix that, we would like to “punish” the model for using higher powers of x .

Lets use the matrix :

$$A = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 2^4 & 0 & 0 \\ 0 & 0 & 3^4 & 0 \\ 0 & 0 & 0 & 4^4 \end{pmatrix}$$

In the following way:

$$\mathcal{L}_\lambda(w) = \frac{1}{m} \|Xw - y\|_2^2 + \lambda \|Aw\|_2^2$$

For the cubic in 2.1:

$$\begin{aligned} \mathcal{L}_\lambda(w) &= \frac{1}{m} \|Xw - y\|_2^2 + \lambda \|Aw\|_2^2 = 0 + \lambda \cdot \left\| \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 2^4 & 0 & 0 \\ 0 & 0 & 3^4 & 0 \\ 0 & 0 & 0 & 4^4 \end{pmatrix} \begin{pmatrix} 0 \\ 2 \\ 0 \\ 1 \end{pmatrix} \right\|_2^2 = \lambda(2^{10} + 4^8) \\ &= \lambda 2^{10}(1 + 2^6) \end{aligned}$$

For the quadratic in 2.2:

$$\begin{aligned} \mathcal{L}_\lambda(w) &= \frac{1}{m} \|Xw - y\|_2^2 + \|w\|_2^2 = 0 + \lambda \cdot \left\| \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 2^4 & 0 & 0 \\ 0 & 0 & 2^8 & 0 \\ 0 & 0 & 0 & 2^{12} \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 3 \\ 0 \end{pmatrix} \right\|_2^2 = \lambda \cdot (3 \cdot 2^8)^2 \\ &= \lambda(9 \cdot 2^{16}) = \lambda 2^{10}(3^2 \cdot 2^6) \end{aligned}$$

We know that $\lambda > 0$ so :

$$\lambda 2^{10}(1 + 2^6) < \lambda 2^{10}(3^2 \cdot 2^6)$$

Which means that the model will choose the cubic method, because it has smaller loss. The cubic polynomial is simpler in its coefficients and powers than the quadratic one, and this is the model we choose.

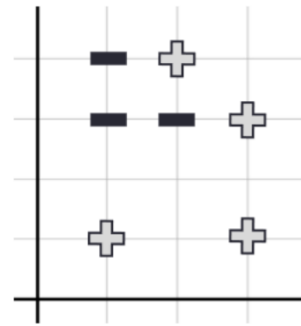
Part C – Boosting

3. Given the following data with binary labels ("+", "-").

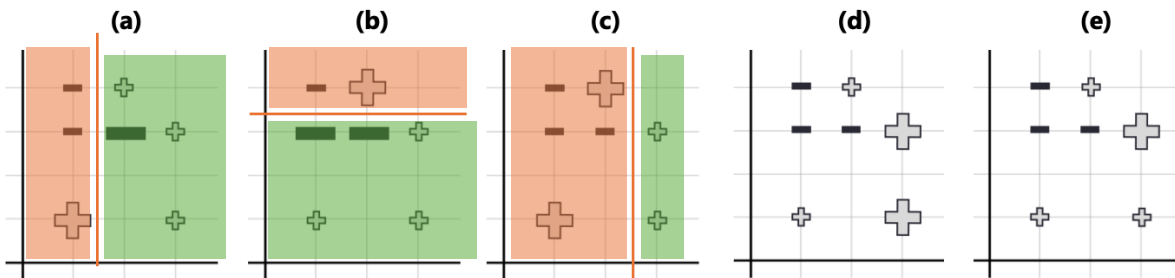
We run AdaBoost with Decision stumps as weak classifiers.

The sizes of the shapes in the figures indicate the probabilities that the algorithm assigns to each sample (high probability = large shape).

Initially, the algorithm starts from a uniform distribution.



Only some of the following figures depict possible distributions that can be obtained after one iteration of AdaBoost. **Which ones?** For each such distribution, propose a weak classifier that can lead to its figure (use a clear drawing or a short description of that classifier).



After just one iteration of AdaBoost, the classifier must be parallel to one of the axes.

lets sign the negative part of space in red and the positive in green.

- (a) Possible – we can see the colors an the classifier on the graph.
- (b) Possible – we can see the colors an the classifier on the graph.
- (c) Possible – we can see the colors an the classifier on the graph.
- (d) Impossible – the pluses in the right side of the graph are bigger than in step 0. That means that the classifier on the first step is wrong on their prediction, and they are found in red space. On the other hand, we have the minuses that are smaller than in step 0, so the classifier was right on them, do they are also in red place. But the other pluses are smaller than step 0 which mans they need to be in green space. There is no classifier that is parallel to one of the axes that can separate the space like this.
- (e) Impossible – the pluses in the right side of the graph are bigger than in step 0. That means that the classifier on the first step is wrong on their prediction, and they are found in red space. Because the classifier is just one line that parallel to one of the axes, the red space should be or under and above the big plus by line that parallel to the left axe, and that cant happen because it was right on the small plus under the bigger one – whish mean it in green space. In the same way can't be a classifier that is parallel to the bottom axe, because of the small pluses that the model has to be right on them an place them in a green space.

4. We have informally argued that the AdaBoost algorithm uses the weighting mechanism to “force” the weak learner to focus on the problematic examples in the next iteration. In this question we will find some rigorous justification for this argument.

Show that the error of h_t w.r.t the distribution D^{t+1} is exactly $1/2$. That is, show that $\forall t \in [T]$

$$\sum_{i=1}^m D_i^{t+1} \mathbb{I}_{[y_i \neq h_t(x_i)]} = 1/2.$$

$$D_i^{t+1} = \frac{D_i^t e^{(-\alpha_t y_i h_t(x_i))}}{Z_t}, \quad Z_t = \sum_{i=1}^m D_i^t e^{(-\alpha_t y_i h_t(x_i))}$$

$$\begin{aligned} \sum_{i=1}^m D_i^{t+1} \cdot \mathbb{I}_{[y_i \neq h_t(x_i)]} &= \sum_{i=1}^m \frac{D_i^t e^{(-\alpha_t y_i h_t(x_i))}}{Z_t} \cdot \mathbb{I}_{[y_i \neq h_t(x_i)]} \\ &= \sum_{i=1}^m \frac{D_i^t e^{(-\alpha_t y_i h_t(x_i))}}{\sum_{j=1}^m D_j^t e^{(-\alpha_t y_j h_t(x_j))}} \cdot \mathbb{I}_{[y_i \neq h_t(x_i)]} \\ &= \sum_{i \text{ s.t. } y_i \neq h_t(x_i)} \frac{D_i^t e^{(-\alpha_t y_i h_t(x_i))}}{\sum_{j=1}^m D_j^t e^{(-\alpha_t y_j h_t(x_j))}} \cdot 1 + \sum_{i \text{ s.t. } y_i = h_t(x_i)} \frac{D_i^t e^{(-\alpha_t y_i h_t(x_i))}}{\sum_{j=1}^m D_j^t e^{(-\alpha_t y_j h_t(x_j))}} \cdot 0 \\ &= \sum_{i \text{ s.t. } y_i \neq h_t(x_i)} \frac{D_i^t e^{(-\alpha_t(-1))}}{\sum_{j=1}^m D_j^t e^{(-\alpha_t y_j h_t(x_j))}} \\ &= \sum_{i \text{ s.t. } y_i \neq h_t(x_i)} \frac{D_i^t e^{(-\alpha_t(-1))}}{\sum_{j \text{ s.t. } y_j \neq h_t(j)} D_j^t e^{(-\alpha_t(-1))} + \sum_{j \text{ s.t. } y_j = h_t(j)} D_j^t e^{(-\alpha_t \cdot 1)}} \\ &= \sum_{i \text{ s.t. } y_i \neq h_t(x_i)} \frac{D_i^t e^{(\alpha_t)}}{\sum_{j \text{ s.t. } y_j \neq h_t(j)} D_j^t e^{(\alpha_t)} + \sum_{j \text{ s.t. } y_j = h_t(j)} D_j^t e^{(-\alpha_t)}} \\ &= \sum_{i \text{ s.t. } y_i \neq h_t(x_i)} \frac{D_i^t}{\left(\sum_{j \text{ s.t. } y_j \neq h_t(j)} D_j^t e^{(\alpha_t)} + \sum_{j \text{ s.t. } y_j = h_t(j)} D_j^t e^{(-\alpha_t)} \right) e^{(\alpha_t)}} \\ &= \sum_{i \text{ s.t. } y_i \neq h_t(x_i)} \frac{D_i^t}{\left(\sum_{j \text{ s.t. } y_j \neq h_t(j)} D_j^t + \frac{\sum_{j \text{ s.t. } y_j = h_t(j)} D_j^t}{e^{(2\alpha_t)}} \right)} = (*) \end{aligned}$$

Now notice that we know that the sum of all the distributions is 1.

$$\begin{aligned}
\epsilon_t &= \sum_i D_i^t \cdot \mathbb{1}_{[y_i \neq h_t(x_i)]} = \sum_{j \text{ s.t. } y_j \neq h_t(j)} D_j^t = \frac{\sum_{j \text{ s.t. } y_j \neq h_t(j)} D_j^t}{\sum_{j \text{ s.t. } y_j \neq h_t(j)} D_j^t + \sum_{j \text{ s.t. } y_j = h_t(j)} D_j^t} \\
\Rightarrow \frac{\sum_{j \text{ s.t. } y_j \neq h_t(j)} D_j^t}{\epsilon_t} - \sum_{j \text{ s.t. } y_j \neq h_t(j)} D_j^t &= \sum_{j \text{ s.t. } y_j = h_t(j)} D_j^t \\
\alpha_t &= \frac{1}{2} \log \left(\frac{1}{\epsilon_t} - 1 \right) = \frac{1}{2} \log \left(\frac{1 - \epsilon_t}{\epsilon_t} \right) = \frac{1}{2} \log \left(\frac{1}{\sum_i D_i^t \cdot \mathbb{1}_{[y_i \neq h_t(x_i)]}} - 1 \right) \\
&= \frac{1}{2} \log \left(\frac{1 - \sum_i D_i^t \cdot \mathbb{1}_{[y_i \neq h_t(x_i)]}}{\sum_i D_i^t \cdot \mathbb{1}_{[y_i \neq h_t(x_i)]}} \right)
\end{aligned}$$

So we get:

$$\begin{aligned}
(*) &= \sum_{i \text{ s.t. } y_i \neq h_t(x_i)} \frac{D_i^t}{\left(\sum_{j \text{ s.t. } y_j \neq h_t(j)} D_j^t + e^{(-2\alpha_t)} \sum_{j \text{ s.t. } y_j = h_t(j)} D_j^t \right)} \\
&= \sum_{i \text{ s.t. } y_i \neq h_t(x_i)} \frac{D_i^t}{\left(\sum_{j \text{ s.t. } y_j \neq h_t(j)} D_j^t + \left(\frac{\epsilon_t}{1 - \epsilon_t} \right) (1 - \epsilon_t) \right)} \\
&= \sum_{i \text{ s.t. } y_i \neq h_t(x_i)} \frac{D_i^t}{\left(\sum_{j \text{ s.t. } y_j \neq h_t(j)} D_j^t + \epsilon_t \right)} \\
&= \frac{\sum_{j \text{ s.t. } y_j \neq h_t(j)} D_j^t}{\left(\sum_{j \text{ s.t. } y_j \neq h_t(j)} D_j^t + \sum_{j \text{ s.t. } y_j = h_t(j)} D_j^t \right)} = \frac{1}{2}
\end{aligned}$$

Part D – Perceptron

5. Recall the vanilla perceptron algorithm: For an input trainset $(x_1, y_1), \dots, (x_m, y_m)$

```
w = 0d

while didn't separate trainset
  for i=1 to m
     $\hat{y}_i = \text{sign}(\mathbf{w}^T \mathbf{x}_i)$ 

    if  $y_i \neq \hat{y}_i$ 
      w = w +  $\eta y_i \mathbf{x}_i$ 
```

Prove that $\forall \eta > 0$ the perceptron algorithm will perform the same number of iterations, and will converge to a vector that points to the same direction.

Let $\eta_1, \eta_2 > 0$. If the data is not linear separable, it will converge after $0 < n \in \mathbb{N}$ iterations.

For perceptron with η_1 we get:

$$w_0 = 0, \quad w_i = w_{i-1} + \eta_1 y_i x_i = \sum_{j=1}^i \eta_1 y_j x_j = \eta_1 \sum_{j=1}^i y_j x_j$$

For perceptron with η_2 we get:

$$w_0 = 0, \quad w_i = w_{i-1} + \eta_2 y_i x_i = \sum_{j=1}^i \eta_2 y_j x_j = \eta_2 \sum_{j=1}^i y_j x_j$$

We can see that this is the same vector with a difference of a specific positive constant, which will just shrink or increase the vector size but not its direction. So for every η we will get the same vector direction, and there may be a difference in the sizes.

Lets show we will get same number of iterations.

Let $\eta > 0$.

$$\hat{y}_i = \text{sign}(\langle w_n, x_i \rangle) = \text{sign} \left(\left\langle \eta \sum_{j=1}^n y_j x_j, x_i \right\rangle \right) \underset{\eta > 0}{=} \text{sign} \left(\left\langle \sum_{j=1}^n y_j x_j, x_i \right\rangle \right)$$

By this we can see that $\hat{y}_i$ isn't affected by η , so for every η we will get the same $\hat{y}_i$.